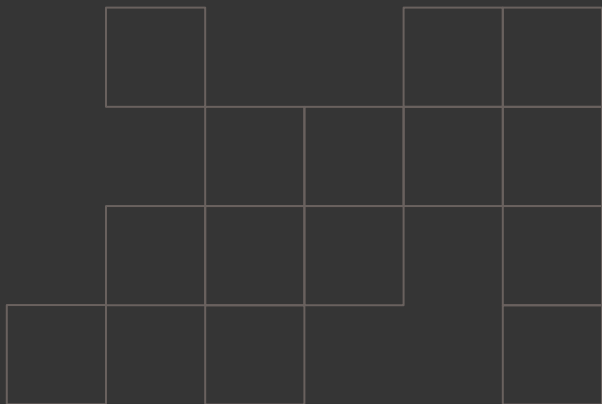


Team Sol |
IIT Mandi |
Edge AI for Personalized
& Connected Vehicles

Falcon

Edge AI Companion for Personalized and Safer In-Car
Experience



Aditya Kumar Singh - Solo team

Who we are

- Solo team **Sol**, led by an engineering student specializing in AI/ML, computer vision, and edge deployment.
- Prior work on autonomous-driving-adjacent projects (sports-tracking vision, RL agents, wake-word models, edge devices like Raspberry Pi).
- Bringing this experience to build **Falcon**, an in-cabin Edge AI companion focused on safety, personalization and privacy.



Full Name
Title

The problem: distracted and generic driving experiences

- Driver distraction and fatigue are involved in a large share of serious crashes; studies attribute around **10–20% of serious road accidents** to fatigue alone, and distraction is officially recorded in **thousands of fatalities per year**.
- Modern vehicles and smartphones push constant notifications and infotainment interactions, increasing visual and cognitive load in critical driving moments.
- Existing in-car systems are mostly **one-size-fits-all**: static alerts, fixed notification behavior, and weak personalization that largely runs in the cloud, limited by connectivity and privacy concerns.

Objective & Approach

Step 1

Use in-cabin camera, microphone, and basic vehicle signals to capture real-time data.

Step 2

Estimate driver attention, fatigue, and context seamlessly on the edge.

Step 3

Learn per-driver habits over time (routes, break patterns, media/climate preferences).

Step 4

A policy engine decides when and how to surface information, alerts, and actions.

Goal

Build Falcon: an on-vehicle AI companion adapting to reduce distraction and improve comfort.

Challenges Faced & Risks

Potential challenge

How we'll respond

1

Robustness across drivers & conditions

Lighting, camera angles, and driver diversity can hurt computer vision accuracy.

Use data augmentation, transfer learning from diverse in-cabin datasets, and conservative thresholds to mitigate this challenge.

2

Latency and compute limits

Edge hardware is constrained and real-time processing is required for safety.

Choose lightweight AI architectures, utilize quantization/pruning, and employ frame-skipping strategies for optimal efficiency.

3

Privacy & user acceptance

In-cabin monitoring raises significant trust and data privacy concerns.

Ensure all processing stays strictly on-device, provide clear user controls, and guarantee no raw media (video/audio) leaves the car.

Technical Implementation

Models & training

Core development
utilizing Python and

PyTorch.

Goal: Scalable Dev

Export models to ONNX
formats with quantization
and pruning.

Goal: Efficient Inference

Edge runtime

Deployment using ONNX
Runtime or TensorRT.

Goal: Low Latency

Targeting Nvidia Jetson
or local proxy compute;
strictly edge-based.

**Goal: 100% Edge
Processing**

Data Pipeline & App

Train on public in-cabin datasets
+ synthetic context data. Log
local embeddings only.

Goal: Privacy-First Data

FastAPI/Flask backend with a
web dashboard for live state
visualization.

Goal: Clear Demonstration



Current Status | Planned Results

1

Idea & Analysis

Problem, architecture, metrics and tech stack have been completely defined. Requirement analysis is complete.

2

Stage-1 (Now)

Completed detailed concept, architecture diagrams, and slide deck. Repository and mock dashboard structure setup.

3

Pre Stage-2 POC

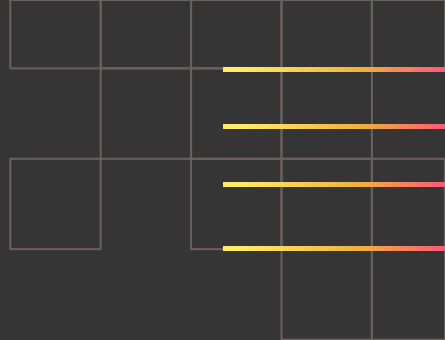
Implement first end-to-end pipeline with webcam input, basic distraction detection, and notification gating demo.

4

Final Demo Phase

Run user tests in simulated driving scenarios, record metrics, and demonstrate measurable reduction in cognitive load and distraction.

Demo Plan | POC Walkthrough



1

Monitor Driver

Use lightweight computer-vision models on in-cabin video to track gaze, eye closure, and head pose for signs of distraction or drowsiness.

2

Listen Context

Process wake words and voice tone locally to detect stress levels and immediate intent without ever sending raw audio to the cloud.

3

Track Variables

Fuse telematics, trip length, time of day, road type, and recent UI interactions into a compact, real-time context embedding.

4

Adapt & Act

Policy engine computes cognitive load to gate/batch notifications, suggest micro-breaks, and auto-apply driver-specific comfort presets.

Future Enhancements & Roadmap

Phase 1

Phase 2

Phase 3

Objectives

Finalize requirements, system architecture, and locate appropriate datasets for model training.

Implement minimal Falcon POC demonstrating core edge capabilities and UI adaptation.

Improve models, add personalization over time, and optimize edge deployment for scale.

Activities

→ Define metrics

→ Set up repository

→ Prepare base data pipeline

→ Train webcam driver models

→ Code notification gating logic

→ Collect demo runs

→ Introduce federated learning

→ Integrate navigation APIs

→ Run extended user testing

Deliverables

→ Concept & Architecture Docs

→ Presentation Deck

→ Mock Dashboard UI

→ End-to-end Pipeline Code

→ Working Local Demo

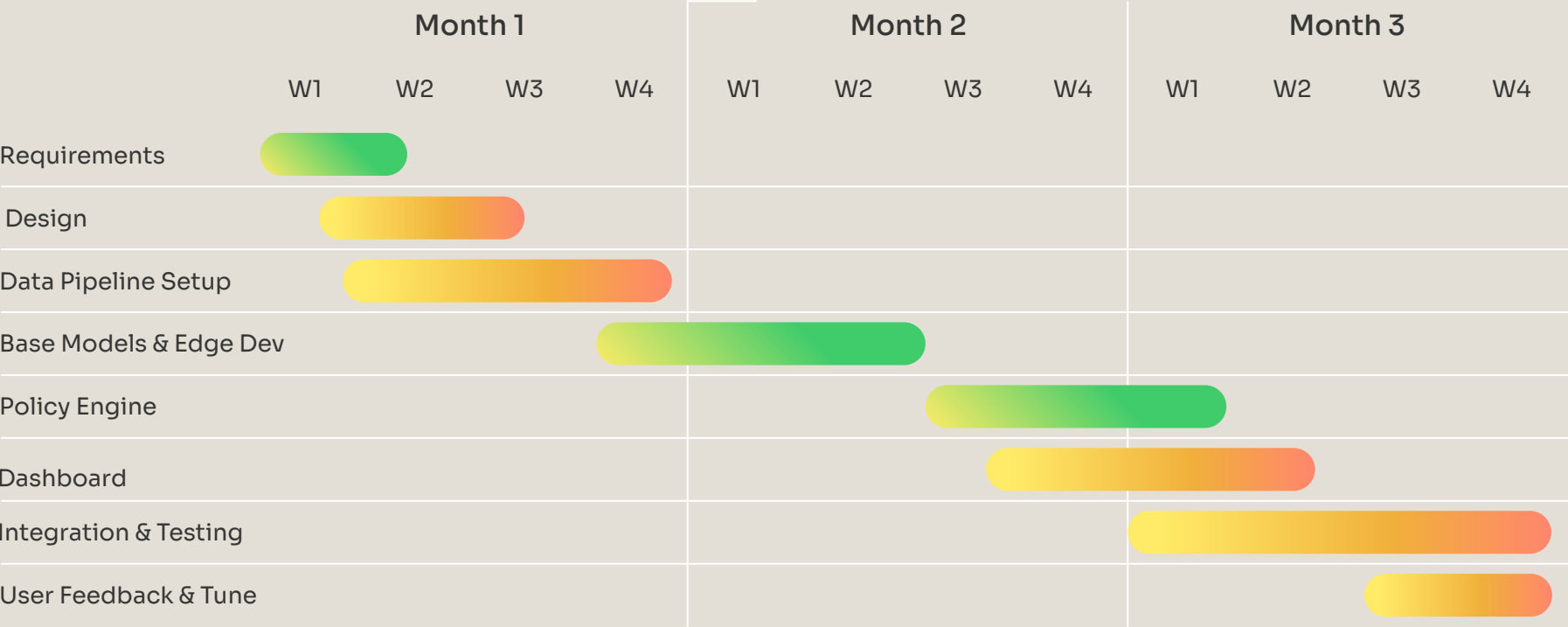
→ Initial Testing Metrics

→ Advanced Edge Models

→ Multi-occupant support

→ Pluggable software layer

Project Timeline





Thank you